

Bienvenue à Initiation à Java

Java : Un langage de programmation puissant

- Créé par **Sun Microsystems** en **1995**, maintenu par **Oracle** depuis 2009
- **Portable** grâce à la JVM, cela signifie que les programmes écrits en Java peuvent être exécutés sur différentes plateformes (Windows, macOS, Linux...) sans modification du code source.
- **Orienté objet** (POO) ce qui facilite la modularité et la réutilisation du code
- **Gestion automatique** de la mémoire (Garbage Collector) (allocation et libération de la mémoire de manière automatique)
- **Sécurisé et fiable**
- **Écosystème riche** (bibliothèques et frameworks)

Types de données en Java

Java dispose de deux types de données, les types primitifs et les types référence.

Les types de référence incluent les classes, tableaux, annotations, interfaces, énumérations, et chaînes de caractères permettant de créer des structures complexes.

On en parlera plus en détails par la suite.

Les types primitifs stockent des valeurs simples (nombres, caractères) et sont très efficaces car ils ont une taille fixe et sont stockés en mémoire.

Il existe 8 types primitifs :

Type	Taille	Plage de valeurs
byte	8 bits	-128 à 127
short	16 bits	-32 768 à 32 767
int	32 bits	-2^{31} à $2^{31}-1$
long	64 bits	-2^{63} à $2^{63}-1$
float	32 bits	Nombre flottant
double	64 bits	Nombre flottant
char	16 bits	Caractère Unicode
boolean	1 bit	true ou false

Opérateurs en Java

- **Arithmétiques** : `+`, `-`, `*`, `/`, `%`
- **Comparaison** : `==`, `!=`, `>`, `<`, `>=`, `<=`
- **Logiques** : `&&`, `||`, `!`
- **Affectation** : `=`, `+=`, `-=`, `*=`, `/=`, `%=`

Déclarer des variables en Java

```
type_variable nom_variable;  
System.out.println("nom de notre variable : " + nom_variable); // permet d'afficher la valeur de la variable dans la console  
// permet d'écrire des commentaires dans le code
```

où `type_variable` est l'un des types de données Java comme `int` ou `float` par exemple et `nom_variable` le nom que l'on choisit.

Une ligne se finie par le `;`.

`System.out.println` permet d'afficher dans la console ce qu'on lui passe en paramètre.

L'opérateur `+` permet ici de faire de la concaténation de chaîne de caractère.

Création d'une classe dans Eclipse

1. **Ouvrir Eclipse** → File > New > Java Project
2. **Nommer le projet** et cliquer sur *Finish*
3. **Créez une package** : Clic droit sur `src` → New > Package
4. **Créez une classe** Clic droit sur `monpackage` → New > Class (ex: `Exercice1`) et cocher `public`
`static void main(String[] args)`
5. **Cliquez sur Finish**
6. **Compilez** clic droit > Run as > Java application

Si vous n'avez pas encore installé votre IDE vous pouvez commencer à coder sur <https://www.online-java.com/>

Création d'une classe dans Vs Code

1. **Créez un projet Java** Ouvrez VsCode > Nouveau Fichier > New Java Project > No build tools
2. **Créez un package** Clic droit > New Java Package
3. **Créez un classe** Sur le package clic droit New Java File > class > Exercice1.java

```
package monpackage;  
public class Exercice1 {  
    public static void main(String[] args) {  
        System.out.println("Hello, World!");  
    }  
}
```

4. **Exécutez** en cliquant sur le bouton Run, le résultat s'affiche dans la console de débogage (Ctrl+Maj+Y).

Pourquoi utiliser des packages ?

- ✓ Organisation propre du projet
- ✓ Évite les conflits de noms
- ✓ Contrôle des accès aux classes
- ✓ Facilite la maintenance et la collaboration
- ✓ Favorise la réutilisation du code

C'est indispensable pour gérer un projet Java propre et efficace !

La méthode **main**

```
public class Exercice1 {  
    public static void main(String[] args) {  
        System.out.println("Hello, Java!");  
    }  
}
```

la méthode **main** est le point d'entrée du programme

Convention de nommage

Par convention on écrira :

les noms de **package** en **minuscules**,

les noms de **classe** en **PascalCase**,

les noms de **méthodes** en **camelCase**.

On utilisera des noms significatifs et on évitera les abréviations inutiles.

Exercice 1 : Déclarer et afficher des variables

Dans la classe Exercice1 :

1. **Déclarez** deux variables `a` et `b` dans la méthode main
2. **Affectez-leur** des valeurs
3. **Effectuez des opérations arithmétiques**
4. **Affichez les résultats** avec `System.out.println()` dans la méthode main

► Afficher le code

Conversion de types

Conversion implicite (widening)

Se produit automatiquement lorsque Java convertit un type plus petit vers un type plus grand (sans risque de perte de données).

```
int a = 10;  
double b = a;  
System.out.println(b); // Affiche 10.0
```

Conversion explicite (casting)

Nécessite un casting manuel ((type) valeur) car elle peut entraîner une perte de données.

```
double x = 9.7;  
int y = (int) x;  
System.out.println(y); // Affiche 9 (partie décimale perdue)
```

Conversion entre String et Types Numériques

De String vers int, double, etc.

```
String str = "123";  
int num = Integer.parseInt(str);  
double d = Double.parseDouble("45.67");  
  
System.out.println(num + 10); // 133  
System.out.println(d + 0.33); // 46.0
```

De int, double, etc. vers String

```
int valeur = 50;  
String texte = String.valueOf(valeur); // "50"  
System.out.println(texte + " est une chaîne !"); // "50 est une chaîne !"
```

Tableaux en Java

Les tableaux classiques (Array)

Un tableau en Java est une structure de données statique qui stocke plusieurs valeurs du même type.

```
// Déclaration et initialisation explicite
int[] nombres = {10, 20, 30, 40, 50};

// Déclaration avec une taille fixe, puis remplissage
String[] noms = new String[3]; // Tableau de taille 3
noms[0] = "Alice";
noms[1] = "Bob";
noms[2] = "Charlie";

// Accéder aux éléments d'un tableau
System.out.println(noms[0]); // Affiche "Alice"
System.out.println(noms[2]); // Affiche "Charlie"
```

Parcourir un tableau

```
for (int i = 0; i < nombres.length; i++) {  
    System.out.println(nombres[i]);  
}  
  
for (String nom:noms){  
    System.out.println(nom);  
}
```

Problème des tableaux fixes :

Une fois la taille du tableau définie, on ne peut pas l'agrandir ou le rétrécir.

Les ArrayList (Listes dynamiques)

```
import java.util.ArrayList;

ArrayList<String> noms = new ArrayList<>();
noms.add("Alice");
noms.add("Bob");
noms.add("Charlie");

System.out.println(noms.get(0)); // Affiche "Alice"
System.out.println(noms.get(2)); // Affiche "Charlie"

noms.add("David"); // Ajoute un élément à la fin
noms.remove(1);    // Supprime "Bob" (index 1)
System.out.println(noms); // Affiche [Alice, Charlie, David]
```

On utilise une ArrayList si on veut pouvoir ajouter ou supprimer des éléments facilement ou si on ne connaît pas la taille exacte à l'avance.

Conditions en Java

```
if (a == b) {  
    System.out.println("la valeur de a est égale à celle de b");  
} else if ("a".equals("a")) {  
    System.out.println("les chaines de caractères sont bien les mêmes");  
} else {  
    System.out.println("cette formation est super");  
}
```

Boucles en Java

Boucle `for`

```
for (int i = 0; i < 10; i++) {  
    System.out.println(i);  
}
```

pour stopper une boucle :

```
for (int i = 0; i < 10; i++) {  
    System.out.println(i);  
    if (i == 8){  
        break;  
    }  
}
```

pour stopper une itération :

```
for (int i = 0; i < 10; i++) {  
    if (i == 8){  
        continue;  
    }  
    System.out.println(i);  
}
```

Boucle `while`

```
int j = 0;
while (j < 10) {
    System.out.println(j);
    j++;
}
```

Boucle `do-while`

```
int k = 0;
do {
    System.out.println(k);
    k++;
} while (k < 10);
```

switch en Java

```
int jour = 3;
switch (jour) {
    case 1:
        System.out.println("Lundi");
        break;
    case 2:
        System.out.println("Mardi");
        break;
    default:
        System.out.println("Autre jour");
}
```

Exercice 2 : Gestion d'un emploi du temps

Pour que votre projet reste clair, créez un nouveau package et une nouvelle classe.

Vous êtes responsable d'un emploi du temps pour une petite équipe. Voici les tâches à réaliser :

- Créer un tableau qui contiendra les noms des 5 membres de l'équipe.
- Afficher les noms des membres de l'équipe à l'aide d'une boucle for.
- Utiliser une condition pour vérifier si un membre spécifique, par exemple "Alice", est présent dans le tableau. Si oui, afficher "Alice est dans l'équipe" ; sinon, afficher "Alice n'est pas dans l'équipe".
- Ajouter un nouveau membre à l'équipe et afficher la liste mise à jour.
- Afficher le nombre total de membres de l'équipe après l'ajout du nouveau membre.

Correction Exercice 2 `EmploiDuTemps`

► Afficher le code

Java : Langage Orientée Objet (POO)

- Java repose sur le concept d'**objets** et de **classes**.
- Facilite la **modularité** et la **réutilisation du code**.
- Basé sur **4 grands principes** :
 - **Encapsulation** 🛡️
 - **Héritage** 👤
 - **Polymorphisme** 🎭
 - **Abstraction** 🎭

Introduction à une Classe en Java

Une **classe** en Java est un modèle (ou un plan) utilisé pour créer des objets.

Elle définit des propriétés et des comportements que les objets de ce type auront.

Elle permet de regrouper des données (variables) et des méthodes (fonctions) sous un même toit,

facilitant ainsi l'organisation du code et l'encapsulation des comportements.

Structure d'une Classe en Java

Une classe peut être vue comme une combinaison de deux éléments principaux :

1. **Attributs (ou variables d'instance)**

Ce sont des propriétés ou des caractéristiques de la classe. Elles représentent l'état de l'objet.

2. **Méthodes (ou fonctions)**

Ce sont des actions ou des comportements que l'objet peut effectuer.
Les méthodes manipulent les attributs et peuvent interagir avec d'autres objets.

Détails Importants sur une Classe en Java

Déclaration d'une classe

- La classe est définie avec le mot-clé `class`.
- Elle peut être publique (public) ou avoir une visibilité par défaut.
 - public : La classe peut être accessible de n'importe quelle autre classe.
 - Visibilité par défaut : La classe est accessible uniquement dans le même package.
- Le nom de la classe se fait en PascalCase par convention.

Exemple de Syntaxe de Base

```
public class NomDeLaClasse {  
    // Attributs (ou variables d'instance)  
    private String nom;  
    private int age;  
  
    // Constructeur (pour initialiser les objets)  
    public NomDeLaClasse(String nom, int age) {  
        this.nom = nom;  
        this.age = age;  
    }  
  
    // Méthodes (ou comportements)  
    public void afficherInfo() {  
        System.out.println("Nom: " + nom + ", Âge: " + age);  
    }  
  
    public void anniversaire() {  
        age++;  
    }  
}
```

Les Getters et Setters

En Java, les getters et setters sont des méthodes d'accès utilisées pour contrôler la lecture et la modification des attributs d'une classe.

- ◆ Getter (get) → Récupère la valeur d'un attribut.
- ◆ Setter (set) → Modifie la valeur d'un attribut avec un contrôle d'accès.

📌 Pourquoi utiliser des getters et setters au lieu d'accéder directement aux variables ?

- ✅ Encapsulation : Protection des données.
- ✅ Validation : Vérification avant modification.
- ✅ Flexibilité : Possibilité d'ajouter de la logique supplémentaire.

Exemple de Base : Sans Getters/Setters

```
public class Personne {  
    public String nom;  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Personne p = new Personne();  
        p.nom = "Alice"; // Accès direct à l'attribut (pas sécurisé)  
        System.out.println(p.nom);  
    }  
}
```

✗ Problèmes :

N'importe qui peut modifier nom sans contrôle.
Impossible d'ajouter des restrictions (ex : empêcher un nom vide).

Implémentation avec Getters et Setters

```
public class Personne {  
    private String nom; // Attribut privé (encapsulation)  
    // Getter : Récupère le nom  
    public String getNom() {  
        return nom;  
    }  
    // Setter : Modifie le nom avec validation  
    public void setNom(String nom) {  
        if (nom == null || nom.trim().isEmpty()) {  
            System.out.println("Erreur : Le nom ne peut pas être vide !");  
        } else {  
            this.nom = nom;  
        }  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Personne p = new Personne();  
        p.setNom("Alice"); // Modification sécurisée  
        System.out.println("Nom : " + p.getNom()); // Lecture sécurisée  
  
        p.setNom(""); // Test d'une valeur invalide  
    }  
}
```

✓ Avantages :

Encapsulation : L'attribut nom est privé.
Validation : Empêche les noms vides.
Flexibilité : Possibilité d'ajouter d'autres contrôles plus tard.

Constructeur

- Le constructeur est une méthode spéciale utilisée pour initialiser un objet lors de sa création.
- Le constructeur porte le même nom que la classe et n'a pas de type de retour (pas même void).
- Si aucun constructeur n'est défini, Java fournit un constructeur par défaut sans paramètres.

```
public class Personne {  
    String nom;  
    int age;  
  
    // Constructeur  
    public Personne(String nom, int age) {  
        this.nom = nom;  
        this.age = age;  
    }  
}
```


Méthodes

- Les méthodes permettent de définir le comportement de la classe. Elles sont utilisées pour manipuler les attributs de l'objet ou effectuer des actions spécifiques.
- Une méthode a un type de retour (par exemple int, String, ou void si aucune valeur n'est retournée).
- Une méthode peut prendre des paramètres et peut également retourner une valeur.

```
public int ajouter(int a, int b) {  
    return a + b;  
}
```

- Les méthodes statiques sont définies avec le mot-clé static et ne dépendent pas d'une instance spécifique de la classe.
 - Elles peuvent être appelées directement sur la classe sans créer d'objet.

Accès aux membres d'une classe

- Les membres de la classe (attributs et méthodes) peuvent avoir des niveaux d'accessibilité variés.
 - `private` : Accès uniquement au sein de la classe.
 - `protected` : Accès dans la même classe, dans le même package, et dans les sous-classes.
 - `public` : Accès dans toutes les autres classes.
 - Visibilité par défaut : Accès dans le même package.
- Le mot-clé `static`
 - Le mot-clé `static` est utilisé pour désigner des membres (attributs ou méthodes) qui sont communs à toutes les instances de la classe.
 - Une méthode ou un attribut statique appartient à la classe elle-même et non à un objet spécifique de la classe.
 - Les attributs statiques peuvent être partagés entre toutes les instances de la classe.

```
public class Main {  
    public int a = 10;  
    public int b = 25;  
  
    public static void main(String[] args) {  
        System.out.println("a+b=" + (a + b)); // Erreur la méthode main ne connaît pas a et b  
    }  
}
```

```
public class Main {  
    public static int a = 10; // Déclaration statique  
    public static int b = 25; // Déclaration statique  
  
    public static void main(String[] args) {  
        System.out.println("a+b=" + (a + b)); // Maintenant, tu peux y accéder directement  
    }  
}
```

```
public class Compteur {  
    static int nombreInstances = 0; // Attribut statique  
  
    public Compteur() {  
        nombreInstances++;  
    }  
}
```

Création d'une instance de classe

- Une instance de classe est un objet créé à partir d'une classe.
- Pour créer une instance, tu utilises le mot-clé `new` suivi du nom du constructeur de la classe.

```
Personne p = new Personne("Alice", 30); // Création d'une instance de la classe Personne
```

- Accès aux membres d'une instance : Une fois l'objet créé, tu peux accéder à ses attributs et méthodes via l'instance.

```
System.out.println(p.nom); // Accède à l'attribut "nom" de l'objet p  
p.afficherDetails();      // Appelle une méthode de l'objet p
```

Exemple d'Utilisation d'une Classe

```
public class Personne {
    // Attributs privés (Encapsulation)
    private String nom;
    private int age;

    // Constructeur
    public Personne(String nom, int age) {
        this.nom = nom;
        this.age = age;
    }

    // Getter pour le nom
    public String getNom() {
        return nom;
    }

    // Setter pour le nom (avec validation)
    public void setNom(String nom) {
        if (nom == null || nom.trim().isEmpty()) {
            System.out.println("Erreur : Le nom ne peut pas être vide !");
        } else {
            this.nom = nom;
        }
    }

    // Getter pour l'âge
    public int getAge() {
        return age;
    }

    // Setter pour l'âge (avec validation)
    public void setAge(int age) {
        if (age < 0) {
            System.out.println("Erreur : L'âge ne peut pas être négatif !");
        } else {
            this.age = age;
        }
    }

    // Méthode pour afficher les informations
    public void afficherInfo() {
        System.out.println("Nom : " + nom + ", Âge : " + age);
    }

    // Méthode pour fêter un anniversaire
    public void anniversaire() {
        age++;
        System.out.println("Joyeux anniversaire, " + nom + "! Vous avez maintenant " + age + " ans.");
    }
}
```

```
public class TestPersonne {  
    public static void main(String[] args) {  
        // Création d'un objet de type Personne  
        Personne personnel = new Personne("Alice", 30);  
  
        // Utilisation des méthodes  
        personnel.afficherInfo(); // Affiche : Nom : Alice, Âge : 30  
        personnel.anniversaire(); // Affiche : Joyeux anniversaire, Alice! Vous avez maintenant 31 ans.  
    }  
}
```

Exercice 3 : Gestion d'une Liste de Personnes

1. Créer une classe Personne avec :

Attributs nom et âge (privés).

Un constructeur.

Getters et Setters (avec validation).

Une méthode afficherInfo().

Une méthode anniversaire() qui augmente l'âge.

2. Créer une classe GestionPersonnes qui :

Contient une liste de plusieurs objets Personne.

Ajoute des personnes à cette liste.

Affiche toutes les personnes.

Permet de fêter l'anniversaire d'une personne spécifique.

Correction Exercice 3 GestionPersonnes

► Afficher le code

Exercice : Gestion d'un Zoo

Vous allez programmer un système de gestion de zoo en Java. Chaque animal du zoo a un âge maximum : lorsqu'il l'atteint, il meurt et est retiré du zoo.

1. Créer une classe Animal

Chaque animal doit avoir les attributs suivants :

```
nom (String) → Nom de l'animal  
espece (String) → Espèce de l'animal  
age (int) → Âge actuel de l'animal  
ageMax (int) → Âge maximum avant sa mort
```

Et les méthodes suivantes :

```
presenter() → Affiche les informations de l'animal  
vieillir() → Augmente l'âge de l'animal de 1 an  
    Si l'animal atteint son âge maximum, il meurt (affichage d'un message et retour false)  
    Sinon, il continue à vivre (retour true)
```

2. Créer une classe Zoo

Le zoo doit contenir une liste d'animaux et permettre :

```
ajouterAnimal(Animal animal) → Ajouter un animal au zoo  
afficherAnimaux() → Afficher tous les animaux encore vivants  
faireVieillirTous() → Faire vieillir tous les animaux du zoo  
Les animaux qui atteignent leur âge maximum sont retirés du zoo
```

3. Créer une classe Main pour tester

Dans cette classe, vous devez :

```
Créer un zoo  
Ajouter plusieurs animaux (Lion, Elephant, Girafe) avec des âges et des âges maximums différents  
Afficher les animaux du zoo  
Simuler plusieurs années avec des appels successifs à faireVieillirTous()  
Afficher l'évolution du zoo au fil des ans
```

Correction Exercice Zoo

► Afficher le code

► Afficher le code

► Afficher le code

Interfaces en Java

- Une **interface** définit un contrat que les classes doivent respecter. Elle permet de spécifier des méthodes que les classes implémentant l'interface doivent définir, mais sans fournir de détail sur la manière dont elles le feront.

```
interface Animal {  
    void faireDuBruit();  
}  
  
class Chien implements Animal {  
    public void faireDuBruit() {  
        System.out.println("Ouaaf Ouaaf !");  
    }  
}
```

Énumérations en Java

Une énumération (ou enum) en Java est un type spécial qui permet de définir un ensemble de constantes (valeurs fixes). Elle est particulièrement utilisée pour représenter des groupes de valeurs liées, telles que les jours de la semaine, les mois de l'année, les couleurs, les directions, etc.

```
enum Jour {  
    LUNDI, MARDI, MERCREDI, JEUDI, VENDREDI, SAMEDI, DIMANCHE;  
}
```

Une fois définie, une énumération peut être utilisée comme un type pour déclarer des variables.
Les valeurs de l'énumération sont des objets de cette énumération.

```
public class TestEnum {  
    public static void main(String[] args) {  
        Jour aujourd'hui = Jour.LUNDI;  
        System.out.println(aujourd'hui);  
    }  
}
```


Encapsulation

- Regroupe **données** (attributs) et **méthodes** (comportements) dans une classe.
- Protège les données avec des **modificateurs d'accès** (`private` , `public` , `protected`).

```
class Personne {  
    private String nom;  
  
    public void setNom(String nom) {  
        this.nom = nom;  
    }  
  
    public String getNom() {  
        return nom;  
    }  
}
```

Exemple concret : Gestion d'un Compte Bancaire

Imagine que tu crées une application bancaire. Un compte bancaire possède un solde, mais on ne doit pas pouvoir modifier directement ce solde depuis l'extérieur.

```
class CompteBancaire {
    private double solde;

    public CompteBancaire(double soldeInitial) {
        this.solde = soldeInitial;
    }

    public void deposer(double montant) {
        if (montant > 0) {
            solde += montant;
            System.out.println("Dépôt réussi. Nouveau solde : " + solde);
        }
    }

    public void retirer(double montant) {
        if (montant > 0 && montant <= solde) {
            solde -= montant;
            System.out.println("Retrait réussi. Nouveau solde : " + solde);
        } else {
            System.out.println("Fonds insuffisants !");
        }
    }

    public double getSolde() {
        return solde;
    }
}
```

Héritage

L'héritage permet à une classe (classe fille) d'hériter des propriétés et méthodes d'une autre classe (classe parent).

Cela favorise la réutilisation du code et la structuration d'un projet.

```
class Animal {  
    void dormir() {  
        System.out.println("Je dors...");  
    }  
}  
  
class Chien extends Animal {  
    void aboyer() {  
        System.out.println("Ouaf Ouaf !");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Chien monChien = new Chien();  
        monChien.dormir(); // Hérité d'Animal  
        monChien.aboyer(); // Spécifique à Chien  
    }  
}
```

Accès aux attributs de la classe parent (super)

Si la classe parent contient des attributs, la classe fille peut les utiliser et même les modifier avec super.

```
class Animal {
    String nom;
    Animal(String nom) {
        this.nom = nom;
    }
    void afficherNom() {
        System.out.println("Je suis " + nom);
    }
}
class Chien extends Animal {
    Chien(String nom) {
        super(nom); // Appelle le constructeur de Animal
    }
    void aboyer() {
        System.out.println(nom + " aboie : Ouaf Ouaf !");
    }
}
public class Main {
    public static void main(String[] args) {
        Chien rex = new Chien("Rex");
        rex.afficherNom();
        rex.aboyer();
    }
}
```

- ✓ Utilisation de super(nom); → Permet à Chien d'appeler le constructeur de Animal et de réutiliser nom.

Surcharge (@Override) d'une méthode

Une classe fille peut modifier le comportement d'une méthode héritée avec l'annotation @Override.

```
class Animal {  
    void faireDuBruit() {  
        System.out.println("L'animal fait un bruit...");  
    }  
}  
  
class Chat extends Animal {  
    @Override  
    void faireDuBruit() {  
        System.out.println("Miaou Miaou !");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Animal a = new Animal();  
        a.faireDuBruit(); // L'animal fait un bruit...  
        Chat c = new Chat();  
        c.faireDuBruit(); // Miaou Miaou !  
    }  
}
```

✓ Pourquoi @Override ?

Il signale que la méthode est réécrite.
Java vérifie que la méthode existe bien dans la classe parent.

Héritage et visibilité (public, protected, private)

Modificateur Accessible dans la classe fille ? Accessible en dehors du package ?

public	✓ Oui	✓ Oui
protected	✓ Oui	⊘ Non (sauf sous-classe)
private	⊘ Non	⊘ Non

```
class Animal {
    protected String espece = "Mammifère"; // Accessible dans les classes filles
    private String secret = "Je suis privé"; // Pas accessible
    void afficher() {
        System.out.println("Espèce: " + espece);
    }
}
class Chien extends Animal {
    void afficherInfos() {
        System.out.println("Je suis un " + espece);
        // System.out.println(secret); ✗ Erreur : secret est privé
    }
}
```

Polymorphisme 🧐

- Une même méthode peut **se comporter différemment** selon la classe qui l'implémente.

```
class Animal {
    void faireDuBruit() {
        System.out.println("Un animal fait du bruit");
    }
}
class Chat extends Animal {
    void faireDuBruit() {
        System.out.println("Miaou !");
    }
}
public class Test {
    public static void main(String[] args) {
        Animal monChat = new Chat();
        monChat.faireDuBruit(); // Affiche "Miaou !"
    }
}
```

Exemple concret : Différentes formes de paiement

Une boutique en ligne propose plusieurs moyens de paiement : Carte Bancaire, PayPal, Crypto.
Ils ont une méthode commune payer().

```
abstract class Paiement {
    abstract void payer(double montant);
}

class CarteBancaire extends Paiement {
    void payer(double montant) {
        System.out.println("Paielement de " + montant + "€ par carte bancaire.");
    }
}

class PayPal extends Paiement {
    void payer(double montant) {
        System.out.println("Paielement de " + montant + "€ via PayPal.");
    }
}

public class Boutique {
    public static void main(String[] args) {
        Paiement paiement1 = new CarteBancaire();
        paiement1.payer(50.0);

        Paiement paiement2 = new PayPal();
        paiement2.payer(25.0);
    }
}
```


Abstraction 🎭

- Définition d'une structure sans imposer d'implémentation spécifique.

```
abstract class Vehicule {  
    abstract void rouler();  
}  
  
class Voiture extends Vehicule {  
    void rouler() {  
        System.out.println("La voiture roule !");  
    }  
}
```

Exemple concret : Formes géométriques

On veut modéliser des formes (cercle, carré, rectangle), mais chaque forme a une manière différente de calculer son aire.

```
abstract class Forme {
    abstract double calculerAire();
}
class Cercle extends Forme {
    private double rayon;
    public Cercle(double rayon) {
        this.rayon = rayon;
    }
    @Override
    double calculerAire() {
        return Math.PI * rayon * rayon;
    }
}
class Rectangle extends Forme {
    private double largeur, hauteur;
    public Rectangle(double largeur, double hauteur) {
        this.largeur = largeur;
        this.hauteur = hauteur;
    }
    @Override
    double calculerAire() {
        return largeur * hauteur;
    }
}
public class TestFormes {
    public static void main(String[] args) {
        Forme f1 = new Cercle(5);
        System.out.println("Aire du cercle : " + f1.calculerAire());
        Forme f2 = new Rectangle(4, 6);
        System.out.println("Aire du rectangle : " + f2.calculerAire());
    }
}
```

© Septembre 2026 - Baptiste Delahais - baptiste.delahais@insee.fr

Pourquoi utiliser la POO en Java ?

- ✓ **Réutilisation du code** : Grâce à l'héritage.
- ✓ **Modularité** : Code organisé en objets réutilisables.
- ✓ **Facilité de maintenance** : Code structuré et évolutif.
- ✓ **Sécurité** : Protection des données avec l'encapsulation.

Les exceptions en Java

Une **exception** est un événement qui interrompt le déroulement normal d'un programme en Java.
Elle survient lorsqu'une erreur se produit à l'exécution.

Pourquoi gérer les exceptions ?

- ✓ Éviter les crashes brutaux
- ✓ Améliorer la lisibilité et la maintenance
- ✓ Gérer proprement les erreurs

Types d'Exceptions

Checked Exceptions (Vérifiées)

- **DéTECTÉES À LA COMPILATION** (ie lors de la conversion du code source en code machine et traduit le code java en bytecode) → Doivent être gérées avec `try-catch` ou `throws`.
- Exemples :
 - `IOException` (Erreur d'entrée/sortie)
 - `SQLException` (Erreur liée aux bases de données)
 - `FileNotFoundException` (Fichier introuvable)

```
import java.io.*;

public class ExempleChecked {
    public static void main(String[] args) {
        try {
            FileReader file = new FileReader("inexistant.txt");
        } catch (FileNotFoundException e) {
            System.out.println("Erreur : fichier introuvable !");
        }
    }
}
```

Unchecked Exceptions (Non Vérifiées)

- **Se produisent à l'exécution** (lorsque le bytecode est interprété et exécuté par la **JVM (Java Virtual Machine)**).
- Exemples :
 - `ArithmeticException` (ex. division par zéro)
 - `NullPointerException` (ex. accès à un objet null)
 - `ArrayIndexOutOfBoundsException` (ex. accès hors limites d'un tableau)

```
public class ExempleUnchecked {  
    public static void main(String[] args) {  
        int[] nombres = {1, 2, 3};  
        System.out.println(nombres[5]); // Erreur : Index hors limites  
    }  
}
```

Gestion des Exceptions en Java

1 try-catch

Permet de capturer et gérer une exception.

```
try {  
    int result = 10 / 0;  
} catch (ArithmeticException e) {  
    System.out.println("Erreur : Division par zéro !");  
}
```

2 finally

Permet d'exécuter un bloc de code quoi qu'il arrive (ex. fermer un fichier, libérer une ressource).

```
try {  
    int x = 10 / 2;  
} catch (ArithmeticException e) {  
    System.out.println("Erreur !");  
} finally {  
    System.out.println("Ce bloc sera toujours exécuté !");  
}
```

3 throws

Permet de propager une exception à une autre méthode.

```
public static void division(int a, int b) throws ArithmeticException {  
    if (b == 0) {  
        throw new ArithmeticException("Division par zéro interdite !");  
    }  
    System.out.println(a / b);  
}  
  
public static void main(String[] args) {  
    division(10, 0); // Lève une exception  
}
```


Création d'une Exception Personnalisée

On peut créer nos propres exceptions en héritant de `Exception` ou `RuntimeException`.

```
class MonException extends Exception {  
    public MonException(String message) {  
        super(message);  
    }  
}  
  
public class Test {  
    public static void main(String[] args) {  
        try {  
            throw new MonException("Erreur personnalisée !");  
        } catch (MonException e) {  
            System.out.println(e.getMessage());  
        }  
    }  
}
```

Exercice 4 : Suivi d'un athlète

1. Créer une classe Athlete

Attributs : nom (String), force (int = 50) doit être privé.

Méthode entraînement() : augmente force de 10.

Méthode manger() : augmente force de 5.

Méthode afficherInfos() : affiche nom et force.

Empêcher qu'un athlète ait une force négative ou dépasse 100

2. Créer une classe AthleteElite (héritage de Athlete)

S'entraîne plus efficacement : entraînement() augmente la force de 20.

Mange mieux : manger() augmente la force de 10.

3. Ajout d'une interface Entraînable :

Contient la méthode entraînement().

Athlete et AthleteElite l'implémentent.

4. Ajout de la méthode repos()

Restaure 5 points de force pour un Athlete.

Restaure 10 points de force pour un AthleteElite.

5. Créer une classe ProgrammeSportif

Instancie un Athlete et un AthleteElite.

Fait s'entraîner et manger les deux athlètes.

Affiche les nouvelles statistiques.

Attention vous devez empêcher la création d'un athlète avec un nom vide ou null et lever une exception personnalisée si le nom est invalide.

Correction Interface **Entrainable**

► Afficher le code

Correction gestion des noms vide ou null

► Afficher le code

Correction : Classe `Athlete`

► Afficher le code

Correction : Classe `AthleteElite` (Héritage)

► Afficher le code

Correction : Classe `ProgrammeSportif`

► Afficher le code

Les Streams

✓ Pourquoi utiliser les Streams ?

```
Écriture plus concise et plus lisible  
Optimisation automatique (parallélisme possible)  
Évite les boucles complexes  
permettent de manipuler des collections de données de manière fluide et optimisée.
```


Exemple d'utilisation des Streams

```
import java.util.Arrays;
import java.util.List;

public class ExempleStream {
    public static void main(String[] args) {
        List<String> noms = Arrays.asList("Alice", "Bob", "Charlie", "David", "Eve");

        // Afficher les noms qui commencent par 'A' en utilisant un Stream
        noms.stream()
            .filter(nom -> nom.startsWith("A"))
            .forEach(System.out::println);
    }
}
```

Ce que fait ce code :

```
Crée une liste de noms.
Filtre les noms qui commencent par "A".
Affiche les résultats directement.
```

A l'Insee

Architecture des applications

Les applications sont généralement organisées sous forme de projets Maven multi-modules, comprenant :

```
Module web : Gère l'interface utilisateur (IHM).  
Module core : Contient la logique métier et les règles de gestion.  
Module batch : Gère les traitements en arrière-plan, comme l'intégration et l'analyse des données.
```

- ▼  octavie [octavie develop]
 - >  ficheBatch
 - >  octavie-batch [octavie develop]
 - >  octavie-core [octavie develop]
 - >  octavie-web [octavie develop]
 - >  sqlEtSchema
 -  pom.xml
 -  readme.md
 -  renovate.json

Technologies utilisées

Les technologies utilisées à l'INSEE sont standardisées pour assurer la maintenance et l'évolutivité des applications :

Technologie	Utilisation
Struts 2	Framework utilisé pour l'IHM des applications existantes, basé sur JSP.
Spring Boot	Utilisé pour les projets plus récents et les migrations , simplifie le développement backend et facilite la maintenance.
Hibernate	Gestion des bases de données relationnelles avec PostgreSQL.
PostgreSQL	Base de données principale des applications.
Maven	Outil de gestion de projet et de dépendances.
Sugoi (API LDAP)	Gestion des droits d'accès et de l'authentification.
Coltrane	Interface pour récupérer les adresses des entreprises.
Java 11 ou 17	Version principale utilisée pour le développement.

Les nouvelles technologies utilisées

Les nouvelles technologies comme Spring Boot, React et Docker sont adoptées pour plusieurs raisons :

✓ Modernisation & maintenance facilitée

Struts et JSP sont anciens et deviennent plus difficiles à maintenir.
Spring Boot simplifie le développement modulaire et scalable.
Les frameworks front-end modernes (React, Angular, Vue) permettent des interfaces plus interactives et réactives.

✓ Performance et scalabilité

Spring Boot est plus léger et rapide que Struts.
Docker + Kubernetes permettent un déploiement optimisé et scalable (moins de crashes en cas de forte charge).

✓ Sécurité et conformité

Struts a connu plusieurs failles de sécurité (ex : CVE Log4j), Spring Boot est plus sécurisé.
Gestion des accès plus fine avec Sudo et Spring Security.

✓ Automatisation & DevOps

CI/CD avec GitLab CI : Automatisation des tests et déploiements.
Conteneurisation avec Docker & Kubernetes pour des mises en production plus fluides.

Exercice 5 : Tickets de maintenance sur l'application SuiviSportif

Jusqu'ici, tout ce que vous avez codé tournait dans une console et partait de zéro. En réalité, à l'INSEE, vous allez rarement créer une application de zéro : vous allez arriver sur du code déjà existant, écrit par quelqu'un d'autre, et devoir y apporter des modifications ciblées à partir d'un ticket métier.

C'est ce qu'on reproduit ici.

SuiviSportif est une petite application web déjà entièrement codée et fonctionnelle (Spring Boot, Tomcat embarqué, Maven), qui enregistre des séances d'entraînement d'athlètes et les affiche dans le navigateur.

Vous ne la créez pas : vous la récupérez, vous la lancez, et vous traitez des tickets dessus.

Lancer l'application

1. Prérequis : JDK 17 et Maven installés (java -version, mvn -version).
2. Ouvrir le ReadMe.md de l'application et suivre les instructions pour la lancer
3. Sur le serveur TomCat qui a démarré sur le port 8080 vous devez voir deux athlètes de démonstration (Alice, Bob) déjà présents avec quelques séances — l'appli n'est pas vide, comme une vraie appli sur laquelle vous arriveriez en poste

En cas de problème au lancement :

- Port 8080 was already in use → changez server.port dans application.properties, ou fermez le processus qui occupe déjà le port.
- Rien ne se passe dans le navigateur → vérifiez dans les logs que la ligne Started SuiviSportifApplication est bien apparue avant d'ouvrir la page.

Ticket #1 : Classer les athlètes par temps total d'entraînement

- Ticket métier : "Sur la page d'accueil, l'entraîneur veut voir les athlètes classés du plus entraîné (temps total cumulé) au moins entraîné, plutôt que dans l'ordre où ils ont été ajoutés."

► Indices

► Correction Ticket #1 — Afficher le code

Ticket #2 : Interdire deux athlètes du même nom

- Ticket métier : "On a eu deux fois 'Alice' dans la liste par erreur, ça fausse le classement. Il faut empêcher la création d'un athlète si un athlète du même nom existe déjà (peu importe la casse), avec un message clair pour l'utilisateur."

► Indices

► Correction Ticket #2 — Afficher le code

Ticket #3 : Des athlètes "élite" qui progressent plus vite

- Ticket métier : "Certains athlètes ont un statut 'élite' : ils gagnent deux fois plus de force à chaque séance que les athlètes classiques. Il faut pouvoir créer ce type d'athlète sans dupliquer tout le code de Athlete."

► Indices

► Correction Ticket #3 — Afficher le code

Ticket #4 : La figure préférée de chaque athlète

- Ticket métier : "Pour personnaliser les conseils d'entraînement, l'entraîneur veut voir, pour chaque athlète, quelle figure il pratique le plus souvent (le plus grand nombre de séances), affichée directement sur la page."

► Indices

► Correction Ticket #4 — Afficher le code

Aller plus loin : la démarche qualité

Rendre l'application la plus qualitative possible. Pour cela on peut :

- augmenter la couverture de test (test unitaire JUnit sur les méthodes)
- ajouter une page accessibilité et rendre l'application accessible
- garder l'application et ses dépendances à jour (maintien en condition de sécurité)

Liens utiles

- Documentation officielle Java : [Java Documentation](#)
- Tutoriels Java : [Java Tutorial](#)
- wiki insee : [Aide Installation Logiciel Insee](#)
- GitHub de la présentation [Présentation Java Initiation](#)

Conclusion

- Java est **puissant, sécurisé et portable**
- **Syntaxe claire**, proche du C/C++
- **POO facilite la modularité**
- **Eclipse simplifie le développement**
- **Pratiquez avec des exercices ! 🚀**

Merci ! 🎉

Des questions ? 🤔